Gorgo Go for Java Programmers

May 31, 2026

Contents

Chapter 12

Context and Concurrency Patterns

Chapters 9 and 10 gave you goroutines, channels, and the `sync` primitives you need to coordinate them. This chapter adds the layer that sits on top of all of that: `context.Context`, the standard way to propagate cancellation and deadlines across goroutine boundaries. It also covers the patterns you will see in real Go services — worker pools, rate limiters, fan-out with error collection, and goroutine leak detection.

context.Context

Java has no direct equivalent to `context.Context`. The closest Java analog is a combination of `Future.cancel()`, `ExecutorService.shutdownNow()`, and a hand-rolled deadline field on a request object — all bolted together differently in every codebase. Go standardizes all of this in a single interface.

```go
// context.Context is defined in the standard library as:
type Context interface {
    Deadline() (deadline time.Time, ok bool) // returns the deadline, if any
    Done() <-chan struct{}                    // closed when work should be cancelled
    Err() error                              // nil until Done is closed; then Canceled or DeadlineExceede
    Value(key any) any                       // returns the value associated with key, or nil
}
```

Every long-running or network-bound function in idiomatic Go accepts a `context.Context` as its first parameter. When the context is cancelled — because a deadline expired, a timeout elapsed, or the caller called a cancel function — `Done()` is closed and `Err()` returns a non-nil error. Your function is expected to notice this and return promptly.

> **Tip:** `context.Context` is not just for HTTP handlers. Use it everywhere work can be cancelled: database queries, RPC calls, file I/O, and long computations.

The Root Contexts

Every context tree starts with one of two roots:

```go
ctx := context.Background() // the default root; never cancelled, no deadline, no values
ctx := context.TODO()       // placeholder for "I haven't wired up a context yet"
```

`context.Background()` is for `main`, top-level servers, and test helpers. `context.TODO()` is a compile-time signal that you know a context should be here but have not plumbed it through yet. Treat `context.TODO()` like a `// TODO` comment — it should not survive into production code.

Cancellation, Deadlines, and Timeouts

The three constructors that add cancellation to a context are `context.WithCancel`, `context.WithDeadline`, and `context.WithTimeout`. Each returns a derived context and a cancel function. **Always call the cancel function**, even if the operation finishes before the deadline. Failing to call cancel leaks the goroutine that the runtime uses to monitor the deadline.

```
// WithCancel returns a copy of parent whose Done channel is closed when cancel is called.
func WithCancel(parent Context) (ctx Context, cancel CancelFunc)

// WithDeadline returns a copy of parent with a deadline set to d.
func WithDeadline(parent Context, d time.Time) (ctx Context, cancel CancelFunc)

// WithTimeout returns WithDeadline(parent, time.Now().Add(timeout)).
func WithTimeout(parent Context, timeout time.Duration) (ctx Context, cancel CancelFunc)
```

Here is a function that fetches a song's lyrics from a slow API, with a two-second timeout:

```
package main

import (
    "context"
    "fmt"
    "time"
)

// fetchLyrics simulates a slow network call; it respects ctx cancellation.
func fetchLyrics(ctx context.Context, song string) (string, error) {
    done := make(chan string, 1)
    go func() {
        time.Sleep(3 * time.Second) // simulate slow work
        done <- "I can't keep loving you the way I do"
    }()
    select {
    case lyrics := <-done:
        return lyrics, nil
    case <-ctx.Done():
        return "", ctx.Err() // context.DeadlineExceeded
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel() // always call cancel

    lyrics, err := fetchLyrics(ctx, "From The Start")
    if err != nil {
        fmt.Println("timed out:", err) // timed out: context deadline exceeded
        return
    }
    fmt.Println(lyrics)
}
```

When the timeout fires, `ctx.Done()` is closed and `ctx.Err()` returns `context.DeadlineExceeded`. If `cancel()` is called before the timeout, `ctx.Err()` returns `context.Canceled`. Your code can distinguish the two with `errors.Is`:

```go
if errors.Is(ctx.Err(), context.DeadlineExceeded) {
    fmt.Println("ran out of time")
}
if errors.Is(ctx.Err(), context.Canceled) {
    fmt.Println("caller gave up")
}
```

Trap: Do not store the cancel function and call it lazily. Use `defer cancel()` immediately after the `WithTimeout` / `WithCancel` call. If you forget, the runtime cannot release resources associated with the context until the parent is cancelled or the program exits.

Checking Cancellation Inside a Loop

Goroutines doing CPU-bound work need to poll the context rather than waiting on a channel:

```go
func processTracks(ctx context.Context, tracks []string) error {
    for _, t := range tracks {
        select {
        case <-ctx.Done():
            return ctx.Err() // bail out early
        default:
        }
        // do real work with t
        fmt.Println("processing:", t)
    }
    return nil
}
```

The `select` with a `default` branch is non-blocking: it drains `Done` if it is already closed but does not block if it is still open. Chapter 10 covered `select` in detail.

context.WithValue

`context.WithValue` attaches a key-value pair to a context that child goroutines can retrieve with `ctx.Value(key)`. This is intended for **request-scoped metadata** — things like a trace ID or an authenticated user — not for passing optional function parameters.

```go
func WithValue(parent Context, key, val any) Context
```

The value is retrieved by calling `Value(key)` on any derived context. If no value is found at the current level, Go walks up the context tree until it finds one or reaches the root.

Use Unexported Key Types

If you use a plain string as a key, any package in the call tree can accidentally overwrite your value by using the same string key. The idiomatic fix is to define a **package-private type** for your keys. Because the type is unexported, no other package can construct a value of that type, so collisions are impossible.

```go
package requestmeta

// traceKey is unexported; no other package can create a value of this type.
type traceKey struct{}

func WithTraceID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, traceKey{}, id)
```

```go
}

func TraceID(ctx context.Context) (string, bool) {
    id, ok := ctx.Value(traceKey{}).(string)
    return id, ok
}
```

Contrast this with the anti-pattern:

```go
// ANTI-PATTERN: string keys can collide across packages
ctx = context.WithValue(ctx, "traceID", "abc-123")
ctx = context.WithValue(ctx, "traceID", "xyz-999") // silently shadows the first one!
```

> **Trap:** Never use a built-in type (string, int, etc.) as a context key. The `go vet` tool will warn you: `should not use built-in type string as key for value.` Always define an unexported struct type for context keys.

Context as First Parameter

Go has a universal convention: if a function accepts a context, it is **always the first parameter** and it is **always named `ctx`**. [*ctx-for-context*]

```go
// idiomatic
func SearchSongs(ctx context.Context, query string) ([]Song, error)

// wrong: context buried in the middle
func SearchSongs(query string, ctx context.Context) ([]Song, error)
```

This convention applies throughout the standard library, all major frameworks, and community packages. Do not put a context inside a struct (except when constructing a long-lived object like an HTTP server); pass it explicitly on each call.

> **Trap:** Do not store a `context.Context` in a struct field and use it later. Contexts are request-scoped. A stored context will be cancelled at unpredictable times. Pass the context explicitly to every function that needs it.

The convention is so consistent that when you see a function signature like `func Foo(ctx context.Context, ...)` you immediately know it can be cancelled, timed out, and carries request metadata. Java has no equivalent signal at the call site.

errgroup — Fan-Out with Error Collection

Chapter 11 showed `sync.WaitGroup` for fan-out. `WaitGroup` works, but it cannot collect errors from goroutines. The `golang.org/x/sync/errgroup` package solves both problems: it waits for a group of goroutines to finish **and** returns the first non-nil error any of them produced. If you pass it a context, it also cancels that context as soon as any goroutine returns an error.

```go
import "golang.org/x/sync/errgroup"

// Group is created with errgroup.WithContext.
// g.Go(f) launches f in a goroutine; g.Wait() blocks until all goroutines finish.
// g.Wait() returns the first non-nil error returned by any goroutine.
func WithContext(ctx context.Context) (*Group, context.Context)
```

```go
func (g *Group) Go(f func() error)
func (g *Group) Wait() error
```

Here is a fan-out that fetches three song titles concurrently and cancels all of them if any fails:

```go
package main

import (
    "context"
    "fmt"
    "time"

    "golang.org/x/sync/errgroup"
)

// fetchTitle simulates fetching a song title; slow returns an error.
func fetchTitle(ctx context.Context, id int) (string, error) {
    titles := []string{"From The Start", "Bewitched", "Too Sweet"}
    select {
    case <-time.After(time.Duration(id+1) * 100 * time.Millisecond):
        return titles[id%len(titles)], nil
    case <-ctx.Done():
        return "", ctx.Err()
    }
}

func main() {
    ctx := context.Background()
    g, ctx := errgroup.WithContext(ctx)

    results := make([]string, 3)
    for i := 0; i < 3; i++ {
        i := i // capture loop variable (Go < 1.22)
        g.Go(func() error {
            title, err := fetchTitle(ctx, i)
            if err != nil {
                return err
            }
            results[i] = title
            return nil
        })
    }

    if err := g.Wait(); err != nil {
        fmt.Println("error:", err)
        return
    }
    for _, t := range results {
        fmt.Println(t)
    }
}
```

When any goroutine in the group returns a non-nil error, `errgroup` cancels the shared context. Goroutines that are still running will see `ctx.Done()` closed and should return promptly. `g.Wait()` blocks until all goroutines have returned, then returns the first error.

Tip: `errgroup` is the idiomatic replacement for `sync.WaitGroup` whenever goroutines can fail. If none of them can fail, `sync.WaitGroup` is fine.

Goroutine Leak Detection

A goroutine leak is a goroutine that starts but never exits. Leaks accumulate over the lifetime of a server: each request might leave one goroutine behind, and after thousands of requests you have thousands of idle goroutines consuming memory and scheduler time. Each leaked goroutine also keeps every value it has closed over alive, preventing GC from reclaiming that memory. [***leaked-goroutine-grows-memory***]

The most common cause is a goroutine blocked on a channel send or receive with no path to exit:

```go
// BUG: this goroutine leaks if the caller stops listening on results.
func streamHits(results chan<- string) {
    for {
        results <- "Work Song" // blocks forever if nobody reads
    }
}
```

Every goroutine must have a clear exit path. [***goroutine-must-exit***] The idiomatic exits are: - the function returns naturally, - a `done` channel is closed, - the context passed in is cancelled.

The `go.uber.org/goleak` package detects leaked goroutines in tests:

```go
import (
    "testing"
    "go.uber.org/goleak"
)

func TestMain(m *testing.M) {
    goleak.VerifyTestMain(m) // fails the test suite if any goroutine leaks
}

func TestNoLeak(t *testing.T) {
    defer goleak.VerifyNone(t) // fails this test if a goroutine leaks

    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    done := make(chan struct{})
    go func() {
        defer close(done)
        select {
        case <-ctx.Done(): // goroutine exits when context is cancelled
        }
    }()
    <-done
}
```

`goleak.VerifyTestMain(m)` runs `m.Run()` internally and checks for leaked goroutines after the test suite completes. `goleak.VerifyNone(t)` checks for leaks at the end of a single test function.

Tip: Run `goleak.VerifyTestMain` in every package that uses goroutines. It is cheap, catches real bugs, and forces you to wire up context cancellation correctly.

Trap: A goroutine that loops forever without a `ctx.Done()` or `done` channel check is always a potential leak. Even if your current tests do not expose it, a future caller that cancels the operation will leave it running. Keep goroutine lifetimes simple enough that the exit paths are obvious at a glance. [***obvious-goroutine-lifetimes***]

GOMAXPROCS

Go's scheduler multiplexes goroutines onto OS threads. `GOMAXPROCS` controls how many OS threads the scheduler uses simultaneously. By default it is set to the number of logical CPUs available to the process (i.e., `runtime.NumCPU()`).

You can read or change it at runtime:

```go
import "runtime"

n := runtime.GOMAXPROCS(0) // 0 means "don't change it; just return the current value"
fmt.Println("GOMAXPROCS:", n)

runtime.GOMAXPROCS(4) // limit to 4 OS threads
```

You can also set it via an environment variable before starting the process:

```
GOMAXPROCS=4 ./myserver
```

In container environments (Docker, Kubernetes), the default `GOMAXPROCS` reads the host CPU count, not the container CPU limit. A container with a limit of 2 vCPUs but a 32-core host will start with `GOMAXPROCS=32`, spawning far more OS threads than it needs. The `go.uber.org/automaxprocs` package reads the cgroup CPU quota and sets `GOMAXPROCS` correctly:

```go
import _ "go.uber.org/automaxprocs" // import for side effect; sets GOMAXPROCS from cgroup quota
```

Tip: For most Go services, the default `GOMAXPROCS` is correct on bare metal. In containers, use `automaxprocs` or set `GOMAXPROCS` explicitly in your deployment manifest.

Key Points

- `context.Context` carries cancellation, deadlines, and request-scoped values across goroutine boundaries; Java has no direct equivalent.
- The three context constructors are `WithCancel` (manual cancel), `WithDeadline` (absolute time), and `WithTimeout` (relative duration); always `defer cancel()`.
- Use unexported struct types as context keys to prevent collisions; never use strings or other built-in types as keys.
- The universal convention is `func Foo(ctx context.Context, ...)` — context is always the first parameter, always named `ctx`, never stored in a struct.
- `golang.org/x/sync/errgroup` is the idiomatic fan-out tool when goroutines can fail; it collects the first error and cancels the shared context.
- Every goroutine must have an exit path; use `go.uber.org/goleak` in tests to catch leaks automatically.
- `GOMAXPROCS` defaults to the number of logical CPUs; set it correctly in container environments with `go.uber.org/automaxprocs`.

Exercises

1. **Think about it:** In Java, cancelling an in-flight operation typically means calling `Future.cancel(true)` or interrupting a thread via `Thread.interrupt()`. Describe how Go's `context.Context` model differs from Java's thread-interrupt approach. What are the advantages of passing a context explicitly rather than relying on a thread-level interrupt mechanism? Consider what happens when a Java thread is blocked in a third-party library that does not handle `InterruptedException`, compared to how a Go function using a context-aware library would behave.

2. **What does this print?**

```go
package main

import (
    "context"
    "fmt"
    "time"
)

func work(ctx context.Context, label string) {
    select {
    case <-time.After(500 * time.Millisecond):
        fmt.Println(label, "done")
    case <-ctx.Done():
        fmt.Println(label, "cancelled:", ctx.Err())
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
    defer cancel()

    go work(ctx, "Bewitched")
    go work(ctx, "Too Sweet")
    time.Sleep(400 * time.Millisecond)
    fmt.Println("main done")
}
```

3. **Calculation:** You run a worker pool with `workers = 3` and feed it a slice of 7 tasks. Each task takes exactly 100 ms. Assuming no overhead and perfect parallelism, how many milliseconds does the pool take to complete all 7 tasks? Show your work: how many rounds of 3 concurrent workers are needed and what does each round contribute?

4. **Where is the bug?**

```go
package main

import (
    "context"
    "fmt"
    "time"
)

func fetchData(url string) <-chan string {
    ch := make(chan string)
    go func() {
```

```go
        time.Sleep(2 * time.Second)
        ch <- "result for " + url
    }()
    return ch
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 500*time.Millisecond)
    defer cancel()

    ch := fetchData("https://example.com/songs")
    select {
    case result := <-ch:
        fmt.Println(result)
    case <-ctx.Done():
        fmt.Println("timed out")
    }
}
```

5. **Write a program:** Implement a function `fanOutFetch(ctx context.Context, songs []string) ([]string, error)` that uses `errgroup` to fetch all song titles concurrently. Simulate each fetch with a `time.Sleep` of a random duration between 50 and 150 ms (use `math/rand`). If any fetch takes longer than 300 ms total (enforced by a timeout on the context passed to `fanOutFetch`), the entire operation should be cancelled and an error returned. Print either all results in order or the cancellation error.